# 06장 파이썬 프로그래밍, 어떻게 시작해야 할까?

06장에서는 짧은 스크립트와 함수들을 만들어 본다. 스크립트란 에디터로 작성한 파이썬 프로그램 파일을 말한다. 프로그래밍 감각을 키우는 데 좋은 재료가 될 것이다. 06장의 모든 예제는 대화형 인터프리터가 아닌 에디터로 작성해야 한다.

목차


- 06-1 내가 프로그램을 만들 수 있을까?
- 06-2 3과 5의 배수를 모두 더하기
- 06-3 게시판 페이징하기
- 06-4 간단한 메모장 만들기
- 06-5 숫자 야구 게임 만들기
- 06-6 시저 암호 만들기

# 06-1 내가 프로그램을 만들 수 있을까?

프로그래밍을 막 시작하려는 사람이 가장 먼저 부딪히는 벽은 아마도 다음과 같을 것이다.

**"문법도 어느 정도 알겠고, 책 내용도 대부분 이해된다. 하지만 이 지식으로 도대체 어떤 프로그램을 만들 수 있을까?"**

이럴 때는 '어떤 프로그램을 짜야지'라는 생각보다는 다른 사람들이 만든 프로그램을 자세히 들여다보고 분석하는 것부터 시작해 보자. 그러다 보면 다른 사람들의 생각도 읽을 수 있고, 새로운 아이디어가 떠오를 수도 있다. 여기에서 가장 중요한 것은 자신의 수준에 맞는 소스를 찾는 일이다. 그래서 06장에서는 매우 쉬운 예제부터 시작해 차츰 수준을 높여 실용적인 예제까지 다룰 것이다. 공부한 내용을 어떻게 활용할 것인지는 여러분의 몫이다.

필자는 예전에 프로그래밍을 막 시작한 사람에게 구구단 프로그램을 짜 보라고 한 적이 있다. 쉬운 과제이고 파이썬 문법도 다 공부한 사람이었는데, 프로그램을 어떻게 만들어야 할지 전혀 감을 잡지 못했다. 그래서 필자는 다음과 같은 해결책을 알려 주었다.

**프로그램을 만들려면 가장 먼저 '입력'과 '출력'을 생각하라.**

가령 구구단 프로그램 중 2단을 만든다면, 2를 입력값으로 주었을 때 어떻게 출력되어야 할지 먼저 생각해 보라고 했다. 그래도 잘 그려지지 않는 것 같아 직접 연습장에 적어 가며 설명해 주었다.

- 함수 이름은? gugu로 짓자
- 입력받는 값은? 2
- 출력하는 값은? 2단(2, 4, 6, 8, ..., 18)
- 결과는 어떤 형태로 저장하지? 연속된 자료형이므로 리스트!

독자들도 함께 따라 해 보기 바란다.

**1.** 먼저 에디터를 열고 다음과 같이 입력한다. gugu라는 함수에 2를 입력값으로 주면 result라는 변수에 결괏값을 넣으라는 뜻이다.

```
result = gugu(2)
```

**2.** 이제 결괏값을 어떤 형태로 받을 것인지 고민해 보자. 2단이므로 결괏값은 2, 4, 6, ... 18까지 나올 것이다. 이런 종류의 데이터는 리스트 자료형이 적합하다. 따라서 `result = [2, 4, 6, 8, 10, 12, 14, 16, 18]` 같은 결과를 얻어야겠다는 생각을 먼저 하고 프로그래밍을 시작하는 태도가 필요하다. 이렇게 머릿속에 그림이 그려지기 시작하면 의외로 생각이 가볍게 좁혀지는 것을 느낄 수 있을 것이다.

**3.** 어떻게 만들지 생각해 봤으므로 1번에서 입력한 문장은 지우고 진짜 프로그램을 짜 보자. 일단 이름을 gugu라고 지은 함수를 다음과 같이 만든다.

```
def gugu(n):
    print(n)
```

위와 같은 함수를 만들고 gugu(2)를 실행하면 2를 출력한다. 즉, 입력값으로 2가 잘 들어오는지 확인하는 것이다.

**4.** 이제 결괏값을 담을 리스트를 하나 생성하자. 앞에서 작성한 print(n)은 입력이 잘되는지 확인하기 위한 것이었으므로 지워도 좋다.

```
def gugu(n):
    result = []
```

**5.** 다음으로 result에 2, 4, 6, ... 18을 어떻게 넣어야 할지 생각해 보자. 필자는 다음과 같이 리스트에 요소를 추가하는 append 함수를 사용하기로 했다.

```
def gugu(n):
    result = []
    result.append(n*1)
    result.append(n*2)
    result.append(n*3)
    result.append(n*4)
    result.append(n*5)
    result.append(n*6)
    result.append(n*7)
    result.append(n*8)
    result.append(n*9)
    return result

print(gugu(2))
```

`[실행 결과]`

```
[2, 4, 6, 8, 10, 12, 14, 16, 18]
```

정말 무식한 방법이지만 입력값 2를 주었을 때 원하는 결괏값을 얻을 수 있었다.

**6.** 그런데 앞에서 만든 함수는 반복이 너무 많다. 자세히 보면 `result.append(n*□)`의 □ 위치에 1부터 9까지 숫자만 다르게 들어가 있다. 똑같은 일을 반복할 때는 '반복문'을 사용한다고 했다. 그렇다면 1부터 9까지 출력해 주는 반복문을 만들면 되지 않을까?

대화형 인터프리터를 열고 다음과 같이 테스트해 보았다.

```
>>> i = 1
>>> while i < 10:
...     print(i)
...     i = i + 1
...
1
2
3
4
5
6
7
8
9
```

매우 만족스러운 결과이다. 이제 위 코드를 gugu 함수에 적용해 보자.

**7.** 이상의 생각을 바탕으로 완성한 gugu 함수는 다음과 같다.

```
# gugu.py
def gugu(n):
    result = []
    i = 1
    while i < 10:
        result.append(n * i)
        i = i + 1
    return result
```

다음과 같이 테스트해 보자.

```
print(gugu(2))
```

[실행 결과]

```
[2, 4, 6, 8, 10, 12, 14, 16, 18]
```

사실 gugu 함수는 이런 과정을 거치지 않고도 바로 만들 수 있는 독자들이 많을 것이다. 하지만 더 복잡한 함수를 만들 때는 이렇게 구체적이고 단계적으로 접근하는 방식이 많은 도움이 된다. 프로그래밍을 할 때는 매우 구체적으로 접근해야 머리가 덜 아프다는 것을 기억하자. 자, 이제 다양한 예제를 접해 보며 여러분 나름대로 멋진 생각을 해 보기 바란다.

# 06-2 3과 5의 배수를 모두 더하기

다음 문제를 어떻게 풀면 좋을지 생각해 보자.

```
10 미만의 자연수에서 3과 5의 배수를 구하면 3, 5, 6, 9이다. 이들의 총합은 23이다.
1,000 미만의 자연수에서 3의 배수와 5의 배수의 총합을 구하라.
```

- 입력: 1부터 999까지(1,000 미만의 자연수)
- 출력: 3의 배수와 5의 배수의 총합
- 생각해 볼 것:
    1. 3의 배수와 5의 배수는 어떻게 찾지?
    2. 3의 배수와 5의 배수가 겹칠 때는 어떻게 하지?

이 문제의 핵심 포인트는 2가지다. 첫째는 1,000 미만의 자연수를 구하는 방법이고, 둘째는 3과 5의 배수를 구하는 것이다. 이 2가지만 해결하면 문제는 쉽게 풀린다.

**1.** 먼저 1,000 미만의 자연수는 어떻게 구할 수 있을지 생각해 보자. 여러 가지 방법이 있다. 다음과 같이 변수에 초깃값 1을 준 후 루프를 돌리며 1씩 증가시켜서 999까지 진행하는 방법이 가장 일반적이다.

```
n = 1
while n < 1000:
    print(n)
    n += 1
```

또는 다음과 같이 파이썬다운 range 함수를 사용할 수도 있다.

```
for n in range(1, 1000):
    print(n)
```

두 예제 모두 실행하면 1부터 999까지 출력한다.

**2.** 1,000 미만의 자연수를 구하는 방법을 알았으니 이제 3과 5의 배수를 구하는 방법을 알아보자. 1,000 미만의 자연수 중 3의 배수는 다음과 같다.

```
3, 6, 9, 12, 15, 18, …, 999
```

그렇다면 1부터 999까지 수 중에서 3의 배수는 어떻게 알 수 있을까? 3으로 나누었을 때 나머지가 0인 경우가 바로 3의 배수이다. 따라서 다음과 같이 % 연산자를 사용하면 3의 배수를 쉽게 찾을 수 있다.

```
for n in range(1, 1000):
    if n % 3 == 0:
        print(n)
```

5의 배수는 `n % 5`가 0인 수로 구할 수 있다.

**3.** 이러한 내용을 바탕으로 만든 최종 풀이는 다음과 같다.

```
# add_multiple.py
result = 0
for n in range(1, 1000):
    if n % 3 == 0 or n % 5 == 0:
        result += n
print(result)
```

3 또는 5의 배수에 해당하는 수를 result 변수에 계속 더해 주었다.

이 문제에는 함정이 하나 있다. 15처럼 3으로도 5로도 나누어지는 수가 중복으로 더해질 수 있다는 점이다. 위 코드에서는 or 연산자를 사용했기 때문에 15는 한 번만 더해진다. 조건문 `n % 3 == 0 or n % 5 == 0`은 n이 3의 배수이거나 5의 배수일 때 한 번만 참이 되기 때문이다.

다음은 중복 문제를 고려하지 않아 잘못된 결과를 출력하는 예이다.

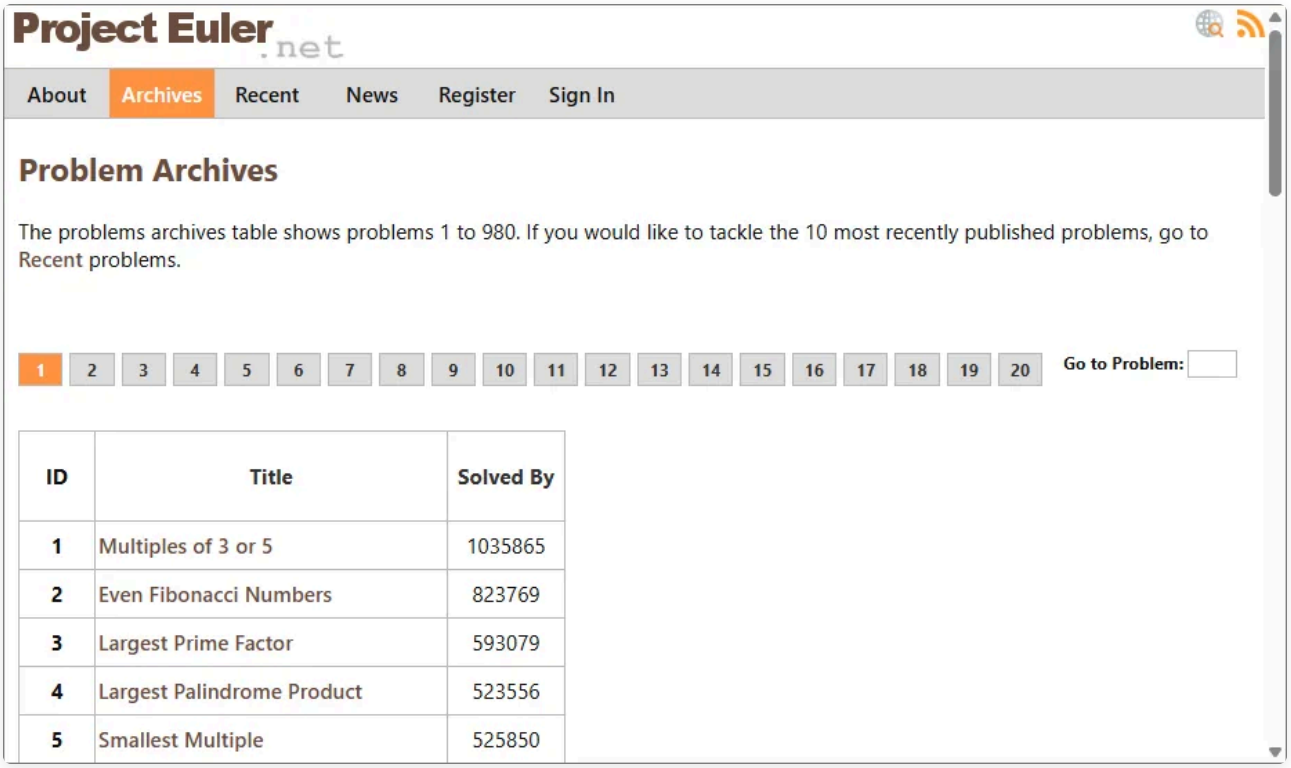
```
result = 0
for n in range(1, 1000):
    if n % 3 == 0:
        result += n
    if n % 5 == 0:
        result += n
print(result)
```

이 코드는 15처럼 3의 배수이면서 5의 배수인 수를 두 번 더하게 되어 잘못된 결과가 나온다.

점프 투 파이썬

**코딩 연습을 할 수 있는 사이트**

이 문제는 코딩 연습 사이트 '프로젝트 오일러'의 첫 번째 문제이다. 이 사이트는 첫 번째 문제부터 차례대로 풀 수 있으며 본인이 작성한 답이 맞는지 즉시 확인할 수도 있다.




프로젝트 오일러(https://projecteuler.net/archives)

# 06-3 게시판 페이징하기

A 씨는 게시판 프로그램을 작성하고 있다. 그런데 게시물의 총 개수와 한 페이지에 보여 줄 게시물 수를 입력받아 총 페이지 수를 출력하는 프로그램이 필요하다고 한다.

> *이렇게 게시판의 페이지 수를 구하는 것을 '페이징'이라고 부른다.*

- 함수 이름은? get_total_page
- 입력받는 값은? 게시물의 총 개수(m), 한 페이지에 보여 줄 게시물 수(n)
- 출력하는 값은? 총 페이지 수

A 씨가 필요한 프로그램을 만들기 위해 입력값과 결괏값이 어떻게 나와야 하는지 먼저 살펴보자. 게시물의 총 개수가 5이고 한 페이지에 보여 줄 게시물 수가 10이면 총 페이지 수는 당연히 1이 된다. 만약 게시물의 총 개수가 15이고 한 페이지에 보여 줄 게시물 수가 10이라면 총 페이지 수는 2가 될 것이다.

| 게시물의 총 개수(m) | 페이지당 보여 줄 게시물 수(n) | 총 페이지 수 |
|---|---|---|
| 5 | 10 | 1 |
| 15 | 10 | 2 |
| 25 | 10 | 3 |
| 30 | 10 | 3 |

이 문제는 게시판 프로그램을 만들 때 가장 처음 마주치는 난관이라 할 수 있다. 실제 업무에서 사용하는 페이징 기술은 훨씬 복잡하지만, 여기서는 가장 간단한 총 페이지 수를 구하는 방법만 알아본다.

**1.** 다음과 같이 총 게시물 개수(m)를 한 페이지에 보여 줄 게시물 수(n)로 나누고 1을 더하면 총 페이지 수를 얻을 수 있다.

```
총 페이지 수 = (총 게시물 개수 / 한 페이지당 보여 줄 개수) + 1
```

**2.** 이 공식을 적용했을 때 총 페이지 수가 표의 값처럼 구해지는지 확인해 보자.

```
def get_total_page(m, n):
    return m // n + 1  # m을 n으로 나눌 때 소수점 아래 자리를 버리기 위해 // 연산자 사용

print(get_total_page(5, 10))    # 1 출력
print(get_total_page(15, 10))   # 2 출력
print(get_total_page(25, 10))   # 3 출력
print(get_total_page(30, 10))   # 4 출력
```

첫 번째, 두 번째, 세 번째 케이스는 공식대로 결과가 출력된다. 하지만 네 번째 케이스는 게시물의 총 개수가 30이고 한 페이지에 보여 줄 개수가 10인데 4가 출력되어 실패했다. 잘 생각해 보자. 총 개수가 30이고 한 페이지에 보여 줄 개수가 10이라면 당연히 총 페이지 수는 3이 되어야 한다.

**3.** 실패 케이스는 총 게시물 수를 한 페이지에 보여 줄 게시물 수로 나눈 나머지가 0일 때 발생함을 알 수 있다. 이 문제를 해결하려면 다음과 같이 코드를 변경해야 한다.

```
# paging.py
def get_total_page(m, n):
    [[MARK]]if m % n == 0:
        return m // n
    else:[[/MARK]]
        return m // n + 1

print(get_total_page(5, 10))
print(get_total_page(15, 10))
print(get_total_page(25, 10))
print(get_total_page(30, 10))  # 3 출력
```

나눈 나머지가 0인 경우는 몫만 반환하고, 그 외의 경우에는 몫에 1을 더하여 반환하도록 변경했다.

프로그램을 실행하면 모든 케이스가 원하는 결과를 출력하는 것을 확인할 수 있다.

# 06-4 간단한 메모장 만들기

원하는 메모를 파일에 저장하고 추가 및 조회가 가능한 간단한 메모장을 만들어 보자.

- 필요한 기능은? 메모 추가하기, 메모 조회하기
- 입력받는 값은? 메모 내용, 프로그램 실행 옵션
- 출력하는 값은? memo.txt

가장 먼저 해야 할 일은 메모를 추가하는 것이다. 다음 명령을 실행했을 때 메모를 추가할 수 있도록 만들어 보자.

```
python memo.py -a "Life is too short"
```

memo.py는 우리가 작성할 파이썬 프로그램 이름이다. -a는 프로그램 실행 옵션이고, "Life is too short"는 추가할 메모 내용이다.

**1.** 먼저 다음과 같이 입력으로 받은 옵션과 메모를 출력하는 코드를 에디터로 작성해 보자.

```
# C:/doit/memo.py
import sys

option = sys.argv[1]
memo = sys.argv[2]

print(option)
print(memo)
```

sys.argv는 프로그램 실행 시 입력된 값을 읽어 들이는 파이썬 라이브러리이다. sys.argv[0]은 파이썬 프로그램 이름인 memo.py이므로 여기서는 필요 없다. sys.argv[1]은 프로그램 실행 옵션, sys.argv[2]는 메모 내용이 된다.

**2.** memo.py를 작성했다면 다음 명령을 수행해 보자.

> *memo.py는 `C:\doit` 디렉터리에 저장한다.*

```
C:\doit>python memo.py -a "Life is too short"
-a
Life is too short
```

입력으로 전달한 옵션과 메모 내용이 그대로 출력되는 것을 확인할 수 있다.

**3.** 이제 입력으로 받은 메모를 파일에 쓰도록 코드를 변경해 보자.

```
# c:/doit/memo.py
import sys

option = sys.argv[1]

[[MARK]]if option == '-a':
    memo = sys.argv[2]
    f = open('memo.txt', 'a')
    f.write(memo)
    f.write('\n')
    f.close()[[/MARK]]
```

옵션이 -a인 경우에만 memo 값을 읽어 memo.txt에 저장한다. 메모는 새로운 내용으로 덮어쓰는 것이 아니라 한 줄씩 추가되어야 하므로 파일 열기 모드를 a(추가 모드)로 했다. 그리고 메모를 추가할 때마다 다음 줄에 저장되도록 줄바꿈 문자( `\n` )도 함께 썼다.

**4.** 이제 다음과 같은 명령을 수행해 보자.

```
C:\doit>python memo.py -a "Life is too short"
C:\doit>python memo.py -a "You need python"
```

그리고 파일에 메모가 정상적으로 저장되었는지 확인해 보자.

```
C:\doit>type memo.txt
Life is too short
You need python
```

> *맥/리눅스에서는 `type memo.txt` 대신 `cat memo.txt` 를 사용한다.*

추가한 메모가 정상적으로 저장된 것을 볼 수 있다.

**5.** 이번에는 작성한 메모를 출력하는 부분을 만들 차례이다. 메모 출력은 다음과 같이 동작하도록 만들어 보자.

```
python memo.py -v
```

메모 추가는 -a 옵션, 메모 출력은 -v 옵션을 사용한다.

이제 메모 출력을 위해 다음과 같이 코드를 변경해 보자.

```
# c:/doit/memo.py
import sys

option = sys.argv[1]

if option == '-a':
    memo = sys.argv[2]
    f = open('memo.txt', 'a')
    f.write(memo)
    f.write('\n')
    f.close()
[[MARK]]elif option == '-v':
    f = open('memo.txt')
    memo = f.read()
    f.close()
    print(memo)[[/MARK]]
```

옵션으로 -v가 들어온 경우 memo.txt를 읽어서 출력한다.

**6.** 코드를 수정한 후 다음과 같은 명령을 수행해 보자.

```
C:\doit>python memo.py -v
Life is too short
You need python
```

입력한 메모가 그대로 출력되는 것을 확인할 수 있다.

# 06-5 숫자 야구 게임 만들기

이번에는 컴퓨터와 대결하는 숫자 야구 게임을 만들어 보자. 숫자 야구는 상대방이 정한 서로 다른 3자리 숫자를 맞히는 게임이다. 숫자를 추측하면 스트라이크와 볼로 힌트를 알려주고, 이 힌트를 바탕으로 정답을 찾아가는 것이 이 게임의 핵심이다.

> *3자리 숫자 중 0은 제외한다.*

게임의 규칙은 다음과 같다.

| 판정 | 조건 |
|---|---|
| 스트라이크 | 숫자와 위치가 모두 맞음 |
| 볼 | 숫자는 맞지만 위치가 다름 |

예를 들어 정답이 `324`일 때 `132`를 입력하면 숫자 3은 있지만 위치가 다르므로 볼, 숫자 2는 있지만 위치가 다르므로 볼이다. 따라서 결과는 0스트라이크 2볼이 된다. 만약 `325`를 입력하면 3과 2는 숫자와 위치가 모두 맞으므로 2스트라이크, 5는 정답에 없으므로 결과는 2스트라이크 0볼이 된다.

- 필요한 기능은? 서로 다른 3자리 난수 생성, 스트라이크/볼 판정, 반복 입력
- 입력받는 값은? 사용자가 추측한 3자리 숫자
- 출력하는 값은? 스트라이크, 볼 개수 (정답이면 축하 메시지와 시도 횟수)

**1.** 먼저 컴퓨터가 서로 다른 3자리 숫자를 만들도록 해 보자.

```
# c:/doit/baseball.py
import random

nums = random.sample(range(1, 10), 3)
print(nums)
```

random.sample 함수는 주어진 범위에서 중복 없이 원하는 개수만큼 뽑아 리스트로 반환한다. `range(1, 10)`은 1부터 9까지의 숫자이므로 0은 포함되지 않는다.

실행하면 다음과 비슷하게 출력된다.

```
[3, 7, 1]
```

**2.** 이제 사용자에게 숫자를 입력받도록 코드를 추가해 보자.

```
# c:/doit/baseball.py
import random

nums = random.sample(range(1, 10), 3)

[[MARK]]question = input("3자리 숫자를 입력하세요: ")
print(question)[[/MARK]]
```

input 함수로 사용자에게 3자리 숫자를 입력받았다. 여기서 주의할 점은 input 함수가 반환하는 값은 문자열이라는 것이다. 예를 들어 "123"을 입력하면 question에는 문자열 "123"이 저장된다.

**3.** 입력받은 문자열을 숫자 리스트로 변환해야 컴퓨터의 숫자와 비교할 수 있다. 다음과 같이 코드를 수정해 보자.

```
# c:/doit/baseball.py
import random

nums = random.sample(range(1, 10), 3)

question = input("3자리 숫자를 입력하세요: ")
[[MARK]]guess = list(map(int, question))
print(guess)[[/MARK]]
```

`map(int, question)`은 문자열 "123"의 각 문자 "1", "2", "3"에 int 함수를 적용하여 정수 1, 2, 3으로 변환한다. 이것을 list로 감싸면 `[1, 2, 3]`이라는 리스트가 된다.

실행하면 다음과 같이 동작한다.

```
3자리 숫자를 입력하세요: 123
[1, 2, 3]
```

**4.** 이제 핵심인 스트라이크와 볼을 판정하는 코드를 작성해 보자.

```
# c:/doit/baseball.py
import random

nums = random.sample(range(1, 10), 3)

question = input("3자리 숫자를 입력하세요: ")
guess = list(map(int, question))

[[MARK]]strike = 0
ball = 0

for i in range(3):
    if guess[i] == nums[i]:
        strike += 1
    elif guess[i] in nums:
        ball += 1

print(f"{strike}스트라이크 {ball}볼")[[/MARK]]
```

for문으로 3자리를 차례대로 비교한다. `guess[i] == nums[i]` 는 같은 위치에 같은 숫자가 있는지 확인하는 것으로 이 경우가 스트라이크이다. `guess[i] in nums` 는 숫자가 정답에 포함되어 있지만 위치가 다른 경우로 이것이 볼이다. elif를 사용했으므로 스트라이크로 판정된 숫자는 볼로 중복 판정되지 않는다.

실행하면 다음과 같이 판정 결과가 출력된다.

```
3자리 숫자를 입력하세요: 123
1스트라이크 1볼
```

**5.** 하지만 지금은 한 번만 추측하고 끝난다. 정답을 맞힐 때까지 반복해서 입력받을 수 있도록 while문을 추가해 보자.

```
# c:/doit/baseball.py
import random

nums = random.sample(range(1, 10), 3)
[[MARK]]count = 0[[/MARK]]

[[MARK]]while True:[[/MARK]]
    question = input("3자리 숫자를 입력하세요: ")
    guess = list(map(int, question))
    [[MARK]]count += 1[[/MARK]]

    strike = 0
    ball = 0

    for i in range(3):
        if guess[i] == nums[i]:
            strike += 1
        elif guess[i] in nums:
            ball += 1

    [[MARK]]if strike == 3:
        print(f"정답! {count}번 만에 맞혔습니다.")
        break[[/MARK]]

    print(f"{strike}스트라이크 {ball}볼")
```

count 변수로 시도 횟수를 세고, 3스트라이크가 되면 축하 메시지와 함께 시도 횟수를 출력한 후 break로 반복을 종료한다. 3스트라이크가 아니면 스트라이크와 볼 개수를 알려주고 다시 입력을 받는다.

**6.** 이제 프로그램을 실행해 보자. 다음은 실행 예시이다.

```
C:\doit>python baseball.py
3자리 숫자를 입력하세요: 123
1스트라이크 0볼
3자리 숫자를 입력하세요: 145
1스트라이크 1볼
3자리 숫자를 입력하세요: 174
1스트라이크 0볼
3자리 숫자를 입력하세요: 415
0스트라이크 2볼
3자리 숫자를 입력하세요: 157
2스트라이크 0볼
3자리 숫자를 입력하세요: 159
3스트라이크 0볼
정답! 6번 만에 맞혔습니다.
```

힌트를 잘 활용하면 보통 7~8번 이내에 정답을 맞힐 수 있다. 직접 실행해서 몇 번 만에 맞힐 수 있는지 도전해 보자.

> ***참고:*** *이 프로그램은 더 발전시킬 수 있다. 예를 들어 3자리가 아닌 숫자를 입력했을 때의 오류 처리, 0 또는 중복된 숫자를 입력했을 때의 안내 메시지 등을 추가할 수 있다.*

# 06-6 시저 암호 만들기

이번에는 문자열을 암호화하는 시저 암호 프로그램을 만들어 보자. 시저 암호는 고대 로마의 율리우스 카이사르(시저)가 사용했다고 알려진 암호 방식으로, 알파벳을 일정한 수만큼 밀어서 다른 글자로 바꾸는 것이다. 예를 들어 A를 3칸 밀면 D가 되고, B는 E가 된다.

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z (원본)
D E F G H I J K L M N O P Q R S T U V W X Y Z A B C (3칸)
```

이 방식대로라면 "PYTHON"은 3칸 밀어 "SBWKRQ"이 된다.

- 필요한 기능은? 알파벳을 원하는 칸 수만큼 밀어 암호화하기
- 입력받는 값은? 문자열, 밀 칸 수
- 출력하는 값은? 암호화된 문자열

**1.** 먼저 알파벳 한 글자를 원하는 칸 수만큼 미는 방법을 생각해 보자. 파이썬에서 문자를 숫자로 바꾸려면 ord 함수를 사용하고, 숫자를 다시 문자로 바꾸려면 chr 함수를 사용한다.

```
>>> ord('A')
65
>>> chr(65)
'A'
>>> chr(65 + 3)
'D'
```

ord('A')는 65이고 여기에 3을 더한 68은 chr(68), 즉 'D'가 된다. 이것이 시저 암호의 핵심 원리이다.

**2.** 이 원리를 이용해 문자열 전체를 암호화하는 코드를 작성해 보자.

```
# c:/doit/caesar.py

word = "PYTHON"
key = 3
result = ""

for char in word:
    result += chr(ord(char) + key)

print(result)
```

실행하면 다음과 같이 출력된다.

```
SBWKRQ
```

for문으로 문자열의 각 글자를 순회하면서 ord로 숫자로 바꾸고, key만큼 더한 후, 다시 chr로 문자로 변환하여 result에 이어 붙였다.

**3.** 그런데 여기에 한 가지 문제가 있다. 만약 'X'를 3칸 밀면 어떻게 될까?

```
>>> chr(ord('X') + 3)
'['
```

'X'에서 3칸 밀면 알파벳 범위를 벗어나 `[` 같은 엉뚱한 문자가 나온다. Z 다음에는 다시 A로 돌아가야 하는데, 지금 코드는 그렇게 동작하지 않는 것이다. 이 문제를 어떻게 해결할 수 있을까?

핵심 아이디어는 알파벳을 0부터 25까지의 숫자로 바꿔 생각하는 것이다. A=0, B=1, C=2, ..., Z=25로 놓으면 알파벳 26개를 숫자로 표현할 수 있다. 이렇게 하면 나머지 연산자 `%`를 사용해 Z를 넘어갈 때 다시 A로 돌아오게 할 수 있다.

단계별로 살펴보자.

먼저 알파벳을 0~25 숫자로 바꾸는 방법이다. ord('A')는 65이므로, ord(char) - ord('A')를 하면 A는 0, B는 1, ..., Z는 25가 된다.

```
>>> ord('A') - ord('A')  # A → 0
0
>>> ord('C') - ord('A')  # C → 2
2
>>> ord('X') - ord('A')  # X → 23
23
```

다음으로 여기에 밀 칸 수를 더하고 26으로 나눈 나머지를 구한다. 26으로 나눈 나머지를 구하면 결과가 항상 0~25 사이가 되므로 알파벳 범위를 절대 벗어나지 않는다.

```
>>> (2 + 3) % 26   # C(2)를 3칸 밀면 → 5 → F
5
>>> (23 + 3) % 26  # X(23)를 3칸 밀면 → 26 % 26 → 0 → A
0
>>> (24 + 3) % 26  # Y(24)를 3칸 밀면 → 27 % 26 → 1 → B
1
```

X(23)에 3을 더하면 26인데, 26 % 26은 0이므로 A가 된다. Y(24)에 3을 더하면 27인데, 27 % 26은 1이므로 B가 된다. 이렇게 Z를 넘어가면 자연스럽게 A부터 다시 시작한다.

마지막으로 이 숫자를 다시 알파벳으로 바꾸려면 ord('A')를 더한 후 chr로 변환하면 된다.

```
>>> chr(0 + ord('A'))   # 0 → A
'A'
>>> chr(5 + ord('A'))   # 5 → F
'F'
```

이 과정을 코드로 정리하면 다음과 같다.

```
# c:/doit/caesar.py

word = "PYTHON"
key = 3
result = ""

for char in word:
    [[MARK]]num = ord(char) - ord('A')       # 알파벳을 0~25 숫자로 변환
    shifted = (num + key) % 26        # 밀고 26으로 나눈 나머지
    result += chr(shifted + ord('A')) # 다시 알파벳으로 변환[[/MARK]]

print(result)
```

```
SBWKRQ
```

결과는 2단계와 동일하지만, 이제 X, Y, Z처럼 끝쪽 알파벳도 올바르게 처리된다.

**4.** 이번에는 코드를 함수로 만들고, 사용자가 직접 문자열과 밀 칸 수를 입력할 수 있도록 완성해 보자.

```
# c:/doit/caesar.py

[[MARK]]def caesar(word, key):
    result = ""
    for char in word:
        num = ord(char) - ord('A')
        shifted = (num + key) % 26
        result += chr(shifted + ord('A'))
    return result

word = input("문자열을 입력하세요: ")
key = int(input("밀 칸 수를 입력하세요: "))

print(f"암호화: {caesar(word, key)}")[[/MARK]]
```

3단계에서 작성한 암호화 로직을 caesar 함수로 분리했다. 이렇게 하면 같은 코드를 여러 번 사용할 수 있어 편리하다.

**5.** 이제 프로그램을 실행해 보자. 다음은 실행 예시이다.

```
C:\doit>python caesar.py
문자열을 입력하세요: HAPPY
밀 칸 수를 입력하세요: 3
암호화: KDSSB
```

그렇다면 암호화된 "KDSSB"를 다시 원래 문자열로 되돌리려면 어떻게 해야 할까? 간단하다. 3칸 밀어서 암호화했으니, -3칸(즉, 반대 방향으로 5칸)을 밀면 원래 문자열이 된다.

```
C:\doit>python caesar.py
문자열을 입력하세요: KDSSB
밀 칸 수를 입력하세요: -3
암호화: HAPPY
```

이처럼 암호화할 때 사용한 칸 수에 마이너스를 붙여 다시 암호화하면 원래 문자열로 복호화할 수 있다. 친구에게 암호화된 문자열과 밀 칸 수만 알려주면, 같은 프로그램으로 비밀 메시지를 주고받을 수 있다.

> ***참고:*** *이 프로그램은 더 발전시킬 수 있다. 예를 들어 A-Z가 아닌 문자를 입력했을 때의 오류 처리, 공백을 입력했을 때 공백을 그대로 유지하는 기능 등을 추가할 수 있다.*